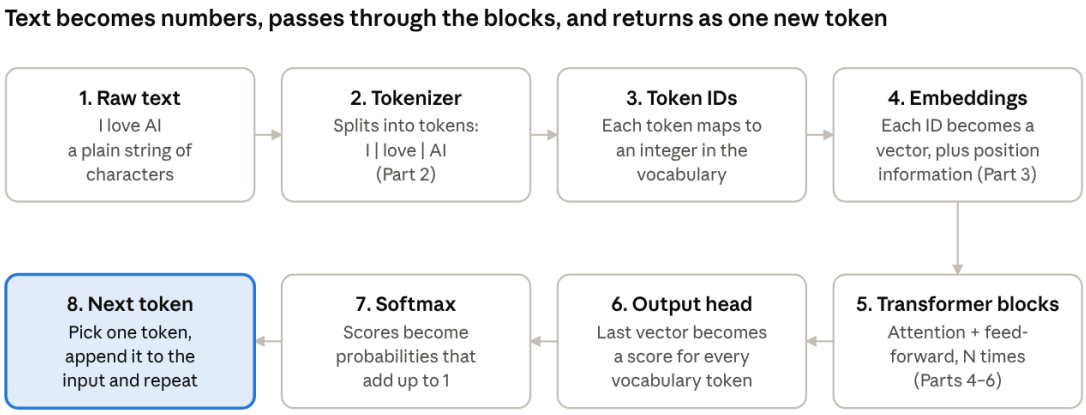
Transformer Architecture: Basics to Detail

Part 1 — Overview: the journey of a sentence

A Transformer never sees words. It converts text into numbers (tokenization and embeddings), mixes those numbers using attention across many layers, and converts the result back into a probability for every possible next token. Introduced in the 2017 paper "Attention Is All You Need", this design underlies GPT, Claude, Gemini, Llama, BERT and most modern AI.



End-to-end data flow · 8 stages in a decoder-only Transformer

Stages 1–4 happen once per input; stages 5–8 repeat for every new token the model writes.

How this report is organised

Part	Topic	Key question answered
2	Tokenization	How is text cut into pieces the model can count?
3	Embeddings and positions	How do those pieces become meaningful numbers?
4–5	Self-attention and multi-head attention	How does each word use context from other words?
6	The Transformer block	What else happens inside each layer?
7	Architectures and scale	How are blocks assembled into BERT, GPT and T5?
8–10	Pretraining vs fine-tuning	How does a model learn, and how is it specialised?
11	Modern refinements	What makes today's models faster and longer-context?

Notation used throughout

Symbol	Meaning	Example (GPT-2 small)
n	Number of tokens in the sequence	up to 1,024
d (d_model)	Size of each token's vector	768
V	Vocabulary size	50,257
h	Number of attention heads	12
d_k	Size per head = d ÷ h	64
L	Number of Transformer blocks (layers)	12

Part 2 — Tokenization

Tokenization splits text into **tokens** — the units a model reads and writes — and maps each to an integer ID from a fixed vocabulary. Modern LLMs use **subword** tokenization: common words stay whole, rare words split into familiar pieces, so any text can be represented with a vocabulary of roughly 30,000–200,000 tokens.

Three ways to split text

Approach	"unbelievable" becomes	Vocabulary size	Pros	Cons
Word-level	[unbelievable]	Huge (millions)	Short sequences; intuitive	Unknown words become [UNK]; can't handle typos or new words
Character-level	[u, n, b, e, l, i, e, v, a, b, l, e]	Tiny (~100s)	Never an unknown word	Very long sequences; meaning spread thin
Subword (used by LLMs)	[un, believ, able]	Medium (30k–200k)	Handles any word; reuses meaningful pieces	Splits can look odd; varies by language

Byte-Pair Encoding (BPE), step by step

BPE, used by GPT models, builds a vocabulary by repeatedly merging the most frequent pair of adjacent symbols in the training text.

Suppose the training text contains the words *low* (×5), *lower* (×2), *newest* (×6) and *widest* (×3).

Step	Most frequent pair	Merge into	Example word after merge
Start	—	—	n e w e s t

Step	Most frequent pair	Merge into	Example word after merge
1	e + s (9 times)	es	n e w es t
2	es + t (9 times)	est	n e w est
3	l + o (7 times)	lo	lo w
4	lo + w (7 times)	low	low
5	n + e (6 times)	ne	ne w est
…	repeat until the vocabulary reaches its target size		

At inference time the tokenizer applies the same merges in order, so an unseen word like *lowest* still tokenizes cleanly as [low, est]. **Byte-level BPE** starts from the 256 raw bytes instead of characters, so emoji, code and any language can always be encoded.

Popular tokenizers

Tokenizer	Algorithm	Used by	Notes
tiktoken (cl100k, o200k)	Byte-level BPE	GPT-3.5, GPT-4, GPT-4o and later	o200k has ~200k tokens and handles non-English text more efficiently
WordPiece	Merges chosen by likelihood, not raw frequency; marks word pieces with ##	BERT	"playing" → [play, ##ing]
SentencePiece (BPE or Unigram)	Treats text as raw characters including spaces (shown as _)	T5, Llama, Gemma, many multilingual models	Language-independent; no pre-splitting on spaces
Unigram LM	Starts large and prunes tokens that least reduce likelihood	T5, ALBERT (via SentencePiece)	Can produce several valid segmentations

Special tokens

Token	Purpose
[CLS], [SEP] (BERT)	Mark the start of input and separate sentence pairs
<|endoftext|>, </s>	Mark the end of a document or reply
[MASK]	Hide a token during masked-language-model training
[PAD]	Fill short sequences to equal length in a batch

Token	Purpose
Chat role tokens	Separate system, user and assistant turns in chat models

Why tokenization matters in practice

- **Cost and limits** are counted in tokens. In English, 1 token ≈ 4 characters ≈ ¾ of a word.
- **Languages differ**: Hindi, Telugu or Tamil text often needs several times more tokens than the same meaning in English, making it slower and costlier.
- **Spelling and arithmetic** are hard because the model sees "strawberry" as a few chunks, not individual letters, and numbers can split unevenly ("12345" → [123, 45]).
- **Leading spaces matter**: " cat" and "cat" are usually different tokens.

Part 3 — Embeddings and positional encoding

An **embedding** turns each token ID into a vector of d numbers (768 in GPT-2 small, 12,288 in GPT-3) that captures the token's meaning. Tokens used in similar contexts end up with similar vectors, so the model can reason about meaning with arithmetic.

The embedding table

The model holds a learnable matrix of size V × d — one row per vocabulary token. Embedding is just a row lookup: token ID 1842 → row 1842. The values start random and are learned during pretraining.

Token ID	Token	Embedding vector (first 4 of d values, illustrative)
0	!	[0.12, −0.48, 0.03, 0.91, …]
1842	love	[−0.22, 0.65, 0.17, −0.04, …]
1843	loved	[−0.19, 0.61, 0.21, −0.09, …]
9552	AI	[0.44, 0.08, −0.71, 0.33, …]

Notice "love" and "loved" have close values — that is what a good embedding learns. GPT-2 small's table alone holds 50,257 × 768 ≈ 38.6 million parameters.

Meaning as geometry

- **Similarity** is measured by cosine similarity: vectors pointing the same way have meanings that are alike.
- **Directions carry meaning**: the classic word2vec example is king − man + woman ≈ queen.
- **Contextual embeddings**: the input embedding of "bank" is the same everywhere, but after a few Transformer layers its vector differs in "river bank" vs "bank loan". This context-sensitivity is what attention adds.

```
\text{cosine}(a, b) = \frac{a \cdot b}{\lVert a \rVert \, \lVert b \rVert}
```

Why positions are needed

Attention treats its input as a set: without extra information, "dog bites man" and "man bites dog" look identical. **Positional encoding** injects word order. The original Transformer adds a fixed sine/cosine pattern to each embedding:

$$PE_{(pos,\,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \qquad PE_{(pos,\,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Each dimension oscillates at a different frequency, giving every position a unique fingerprint and letting the model work out relative distances.

Positional encoding methods compared

Method	How it works	Used by	Strength	Limitation
Sinusoidal (fixed)	Add sine/cosine waves of different frequencies	Original Transformer	No parameters; can in principle extend to longer inputs	Generalises poorly beyond trained lengths in practice
Learned absolute	A trainable vector per position, added to the embedding	BERT, GPT-2, GPT-3	Simple and effective	Hard limit at the maximum trained length
RoPE (rotary)	Rotate query and key vectors by an angle that depends on position	Llama, Mistral, Qwen, many modern LLMs	Encodes relative distance; extends to long context with scaling tricks	Slightly more complex
ALiBi	Subtract a penalty from attention scores that grows with distance	BLOOM, MPT	Extrapolates well to longer inputs	Less common today

Input to the first layer

The first Transformer block receives, for every token, its token embedding plus its position information — a matrix of shape n × d. Every later layer keeps this same shape, refining the vectors as they go.

Part 4 — Self-attention in detail

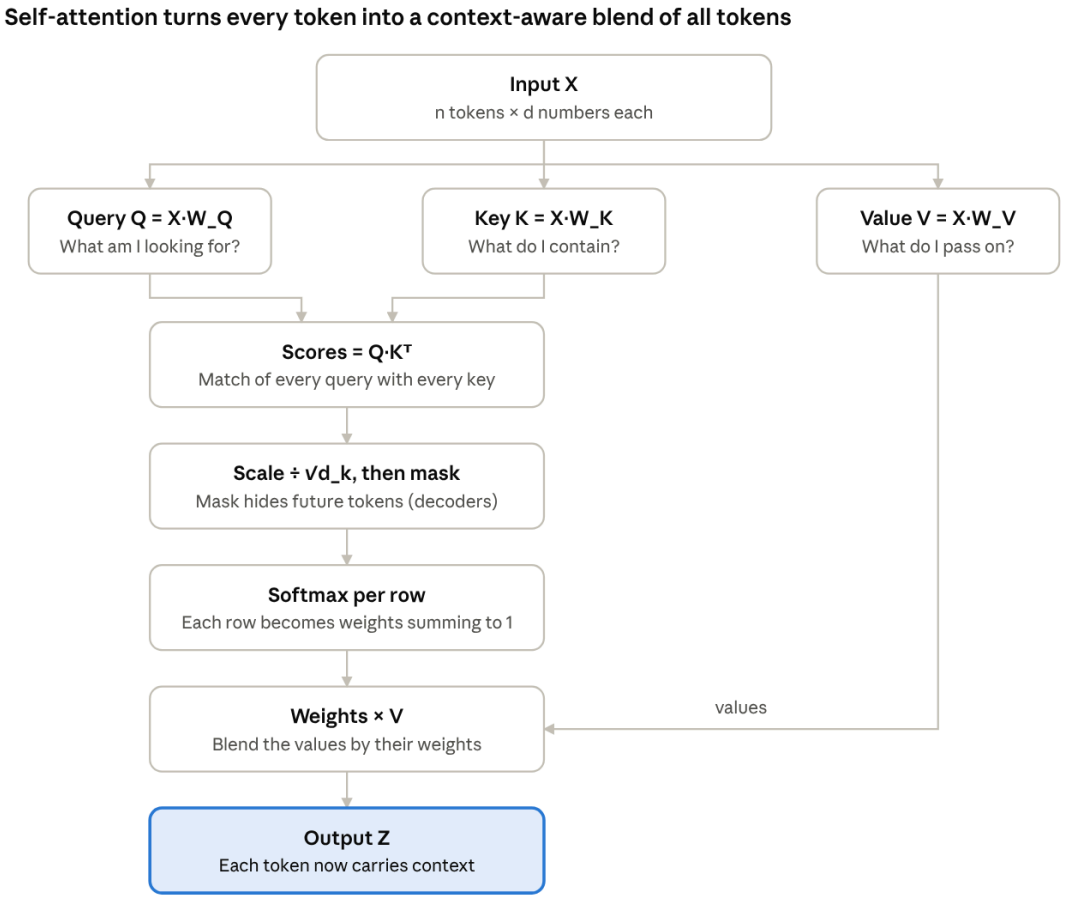
Self-attention lets every token build a new representation of itself by taking a weighted average of all tokens' information, where the weights say how relevant each other token is. It is the one mechanism that separates Transformers from earlier networks.

The formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{T}}{\sqrt{d_k}} + M\right)V$$

Q, K and V are three different projections of the same input; M is the mask (0 where allowed, $-\infty$ where blocked). W_Q, W_K and W_V are learned weight matrices of size d × d_k.

The computation, step by step



Scaled dot-product self-attention · one head

The three projections come from the same input, which is why it is called *self*-attention; in cross-attention (encoder-decoder models), Q comes from the decoder while K and V come from the encoder.

Worked example

Take the three-token input "The cat sat". Focus on the token *sat* and follow its row through the steps (numbers are illustrative).

Step	The	cat	sat	What happened
Raw score q_sat · k	1.2	4.8	2.4	"sat" matches "cat" best: cats sit
Scaled (÷ √64 = 8)	0.15	0.60	0.30	Keeps numbers small so softmax isn't too peaked
Softmax weights	0.27	0.42	0.31	Weights now add to 1.00
Output for "sat"				0.27·v_The + 0.42·v_cat + 0.31·v_sat

The new vector for "sat" is mostly built from "cat", so the model now knows *who* sat. Every token does this in parallel — the whole thing is a few matrix multiplications, which GPUs are very fast at.

Why divide by √d_k?

Dot products grow with vector length. With d_k = 64, raw scores can become large, pushing softmax toward a single 1.0 and many 0.0s, which makes gradients vanish during training. Dividing by √d_k keeps scores in a healthy range.

Causal masking

A decoder must not peek at future tokens while learning to predict them. The mask sets every "future" score to −∞, so after softmax its weight is exactly 0.

Query ↓ / Key →	The	cat	sat
The	✓	×	×
cat	✓	✓	×
sat	✓	✓	✓

Encoders such as BERT use no causal mask, so every token sees the whole sentence in both directions.

Cost

The score matrix is n × n, so compute and memory grow with the **square** of sequence length: doubling the context quadruples the attention work. This is the main reason long context is expensive (Part 11 covers the fixes).

Part 5 — Multi-head attention

Instead of one large attention calculation, a Transformer runs **h smaller attention heads in parallel**, each with its own W_Q, W_K and W_V. Each head can learn to track a different kind of relationship; their outputs are joined and mixed back together.

```
\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\,W_O \qquad
\text{head}_i = \text{Attention}(XW_Q^{i},\, XW_K^{i},\, XW_V^{i})
```

How the sizes work (GPT-2 small)

1. Each token vector has d = 768 values.
2. It is projected into 12 heads, each of size d_k = 768 ÷ 12 = 64.
3. Each head runs the attention from Part 4 independently on its 64-value slices.
4. The 12 outputs (12 × 64 = 768) are concatenated back into one 768-value vector.
5. An output matrix W_O (768 × 768) mixes information across heads.

The total compute is about the same as one big head — multi-head just divides the work so different heads can specialise.

What heads learn (observed in research)

Head type	What it attends to	Example
Previous-token head	The token immediately before	Helps with local word order
Syntactic head	Grammatical partners	Verbs attend to their subjects; adjectives to nouns
Coreference head	What a pronoun refers to	"it" → "the animal"
Induction head	Earlier copies of the current pattern	Seeing "Harry Pot…" again and predicting "ter" — a key driver of in-context learning
Positional / broad head	Spreads attention widely	Gathers overall sentence context

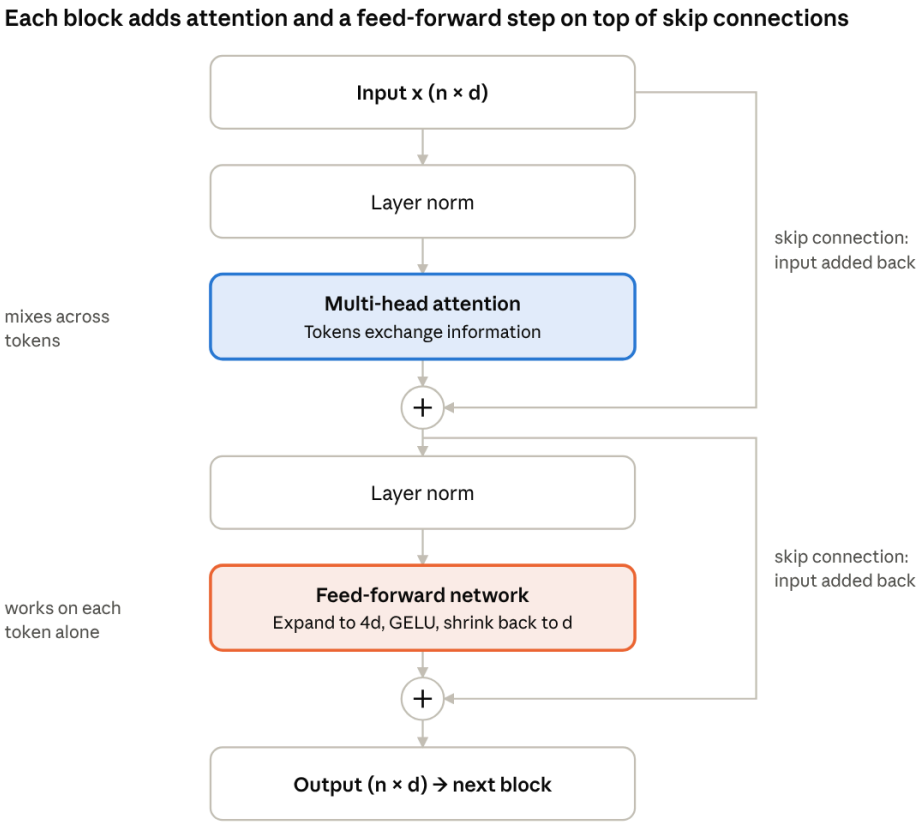
Heads are not assigned roles by humans; these patterns emerge from training. Many heads in large models have no easy interpretation, which is an active research area called *mechanistic interpretability*.

Variants that share keys and values

Variant	Query heads	Key/value heads	Benefit	Used by
Multi-head attention (MHA)	h	h	Most expressive	Original Transformer, GPT-2, BERT
Multi-query attention (MQA)	h	1	Much smaller memory during generation	PaLM, Falcon
Grouped-query attention (GQA)	h	a few groups (e.g. 8)	Near-MHA quality with much less memory	Llama 2 70B and later, Mistral

Part 6 — Inside a Transformer block

A Transformer block has two sub-layers: **attention**, where tokens share information, and a **feed-forward network**, where each token's vector is processed on its own. Each sub-layer is wrapped with layer normalisation and a residual (skip) connection, and the whole block is stacked L times.



One Transformer block · pre-norm layout used by GPT-2, Llama and most modern LLMs

The skip connections mean each sub-layer only has to learn a *change* to its input, which keeps very deep stacks trainable.

The four ingredients

Component	What it does	Why it's needed	Detail
Multi-head attention	Lets each token gather information from other tokens	Context: who did what, what "it" refers to	Parts 4–5
Feed-forward network (FFN / MLP)	Two linear layers with a nonlinearity, applied to each token separately	Stores much of the model's factual knowledge and does per-token "thinking"	d ⟶ 4d ⟶ d; about two-thirds of a block's parameters
Residual connection	Adds the sub-layer's input to its output: x + f(x)	Gradients flow through deep stacks; information is never lost	Also called the "residual stream"

Component	What it does	Why it's needed	Detail
Layer normalisation	Rescales each token vector to a stable mean and spread	Keeps numbers well-behaved for stable training	LayerNorm or the simpler RMSNorm

The FFN in one line

```
\text{FFN}(x) = W_2 \cdot \text{GELU}(W_1 x + b_1) + b_2
```

Modern models often use **SwiGLU**, a gated variant that performs slightly better at the same cost (Llama, Mistral, PaLM).

Pre-norm vs post-norm

	Post-norm (original 2017)	Pre-norm (modern default)
Order	x ⟶ sub-layer ⟶ add ⟶ norm	x ⟶ norm ⟶ sub-layer ⟶ add
Training stability	Needs careful learning-rate warm-up	Much more stable at depth
Used by	Original Transformer, BERT	GPT-2, GPT-3, Llama, most LLMs

After the last block

A final layer norm is applied, then the **output head** — a linear layer of size d × V — turns each token's final vector into V scores (logits). Many models tie this matrix to the input embedding table to save parameters.

Part 7 — Full architectures and scale

The same block from Part 6 is assembled in three ways. Decoder-only has become the dominant design for generative AI because one simple objective — predict the next token — scales to almost any task.

	Encoder-only	Decoder-only	Encoder-decoder
Attention type	Bidirectional (sees all tokens)	Causal (sees only earlier tokens)	Encoder bidirectional; decoder causal + cross-attention
Pretraining objective	Masked language modelling	Next-token prediction	Span corruption / denoising
Output	A rich vector per token	Generated text	Generated text conditioned on input
Best for	Classification, search, embeddings, named-entity tagging	Chat, writing, code, reasoning, agents	Translation, summarisation, structured rewriting

	Encoder-only	Decoder-only	Encoder-decoder
Examples	BERT, RoBERTa, DeBERTa	GPT series, Claude, Llama, Mistral, Gemini	T5, BART, original Transformer

Model sizes compared

Model	Year	Type	Layers (L)	d_model	Heads	Parameters	Context (tokens)
Llama 3.1 70B	2024	Decoder	80	8,192	64 (8 KV groups)	70B	128k
Llama 2 70B	2023	Decoder	80	8,192	64 (8 KV groups)	70B	4k
GPT-3	2020	Decoder	96	12,288	96	175B	2k
T5-base	2019	Encoder-decoder	12 + 12	768	12	220M	512
GPT-2 XL	2019	Decoder	48	1,600	25	1.5B	1k
GPT-2 small	2019	Decoder	12	768	12	124M	1k
BERT-base	2018	Encoder	12	768	12	110M	512

Most frontier closed models (GPT-5 and later, Claude, Gemini) do not publish these figures.

A rule of thumb for parameter count

Each block holds about $4d^2$ parameters in attention (W_Q, W_K, W_V, W_O) and $8d^2$ in the FFN ($d \to 4d \to d$), so:

```
\text{Parameters} \approx 12 \cdot L \cdot d^{2} \; + \; V \cdot d
```

Check with GPT-3: $12 \times 96 \times 12{,}288^2 \approx 174$ billion, plus about 0.6 billion for embeddings — matching its published 175B.

Part 8 — Pretraining

Pretraining teaches a randomly initialised Transformer general language skill and world knowledge by having it predict missing or next tokens across a huge unlabelled corpus. It is **self-supervised**: the text itself supplies the answers, so no human labelling is needed.

Pretraining objectives

Objective	The task	Example	Used by
Causal language modelling (CLM)	Predict the next token from all earlier tokens	"The cat sat on the ___" → mat	GPT, Llama, Claude (decoder-only)
Masked language modelling (MLM)	Predict ~15% of tokens hidden with [MASK], using both sides	"The [MASK] sat on the mat" → cat	BERT, RoBERTa
Span corruption	Replace spans with placeholders; generate the missing spans	"The <X> on the mat" → <X> cat sat	T5
Next-sentence prediction	Does sentence B follow sentence A?	Yes / No	Original BERT (later dropped)

The training loop

1. **Sample a batch** of text sequences (often millions of tokens per batch).
2. **Forward pass**: run them through the model to get a predicted probability for every next token.
3. **Compute the loss**: how surprised the model was by the real next tokens (cross-entropy).
4. **Backward pass**: backpropagation computes how each parameter contributed to the error.
5. **Update** every parameter slightly with an optimiser such as AdamW.
6. **Repeat** for hundreds of thousands of steps until the data budget is used.

$$\mathcal{L} = -\frac{1}{n}\sum_{t=1}^{n} \log P_\theta(x_t \mid x_1, \ldots, x_{t-1})$$

Lower loss means the model assigns higher probability to what actually comes next. **Perplexity** = e^loss is often reported: a perplexity of 10 means the model is as unsure as choosing among 10 equally likely tokens.

What goes into the data

Source	Teaches
Filtered web pages (e.g. Common Crawl)	Broad language and general knowledge
Books	Long-range coherence and narrative
Code (e.g. GitHub)	Programming and structured reasoning
Scientific papers and math	Technical knowledge and step-by-step reasoning
Wikipedia and reference sites	Reliable factual knowledge
Multilingual text	Other languages
Synthetic data from earlier models	Targeted skills, increasingly common

Data quality matters as much as quantity: deduplication, removing spam and toxic content, and filtering out test benchmarks are standard steps.

Scale and compute

Model	Training tokens	Tokens per parameter
Llama 3 70B (2024)	~15 trillion	~214
Chinchilla 70B (2022)	1.4 trillion	20
GPT-3 175B (2020)	~300 billion	~1.7

DeepMind's **Chinchilla** study (2022) showed GPT-3-era models were undertrained: for a fixed compute budget, about 20 tokens per parameter is optimal. Modern models train far beyond that so the final model is smaller and cheaper to run. Frontier pretraining runs use tens of thousands of GPUs for months.

What pretraining produces

A **base model** that is excellent at continuing text but not at following instructions. Ask it "What is the capital of France?" and it may continue with more quiz questions instead of answering — fine-tuning (Part 9) fixes this.

Part 9 — Fine-tuning

Fine-tuning continues training a pretrained model on a smaller, task-specific dataset so it adopts a behaviour, format or domain. Pretraining provides knowledge; fine-tuning shapes how that knowledge is used.

Types of fine-tuning

Method	Data needed	What changes	Typical goal
Supervised fine-tuning (SFT) / instruction tuning	Thousands to hundreds of thousands of prompt → ideal-response pairs	All or some weights	Follow instructions; chat format; a specific task
RLHF (Reinforcement Learning from Human Feedback)	Human rankings of model answers	Weights, guided by a learned reward model	Helpful, harmless, honest behaviour
DPO (Direct Preference Optimisation)	Pairs of chosen vs rejected answers	Weights, directly from preferences	Same goal as RLHF, simpler and more stable
RL on verifiable rewards	Problems with checkable answers (math, code tests)	Weights	Stronger step-by-step reasoning
Domain-adaptive pretraining	Large unlabelled domain text (legal, medical)	All weights	Domain vocabulary and knowledge

Method	Data needed	What changes	Typical goal
Parameter-efficient fine-tuning (PEFT)	Same as SFT	A tiny add-on set of weights	Cheap customisation on modest hardware

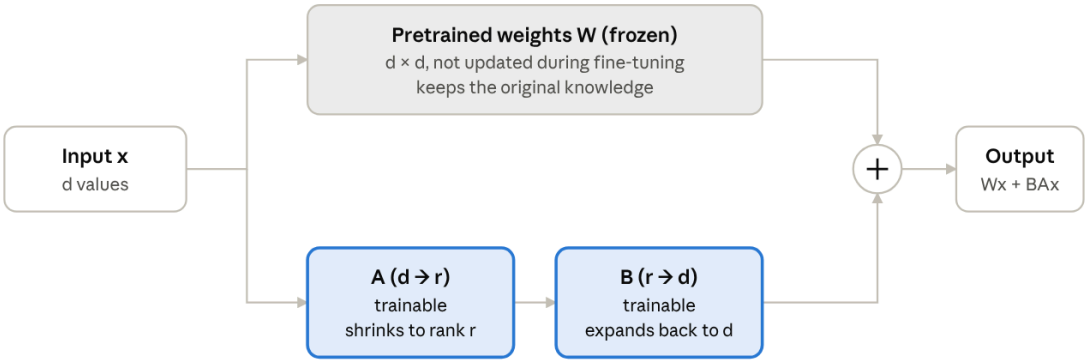
RLHF in three steps

1. **SFT** — fine-tune the base model on demonstrations of good answers.
2. **Reward model** — show humans two or more answers per prompt; train a separate model to predict which they prefer.
3. **Optimise** — use reinforcement learning (typically PPO) to make the model produce answers the reward model scores highly, with a penalty for drifting too far from the SFT model.

Parameter-efficient fine-tuning and LoRA

Full fine-tuning of a 70B model means updating 70 billion parameters and storing optimiser state for each — hundreds of gigabytes of GPU memory. **LoRA** instead freezes the original weights and learns a small low-rank update beside selected matrices (usually the attention projections).

LoRA trains two small matrices beside the frozen weights



r is small (often 8 to 64), so A and B add only 2 × d × r parameters per adapted matrix

LoRA (Low-Rank Adaptation) · frozen path plus trainable low-rank path

Because only A and B are trained, a LoRA adapter is often well under 1% of the model's size, can be swapped in and out, and can be merged into W after training so inference costs nothing extra.

```
h = Wx + \frac{\alpha}{r} BAx
```

PEFT method	Idea	Trainable share
LoRA	Low-rank matrices beside frozen weights	Typically 0.1–1%
QLoRA	LoRA on a base model compressed to 4-bit	Same; lets a 65B model fine-tune on a single 48 GB GPU

PEFT method	Idea	Trainable share
Adapters	Small bottleneck layers inserted in each block	~1–5%
Prefix / prompt tuning	Learn a few "virtual tokens" prepended to the input	<0.1%

Fine-tuning pitfalls

- **Catastrophic forgetting** — aggressive fine-tuning can erase general abilities; use low learning rates, PEFT, or mix in general data.
- **Overfitting** — small datasets get memorised; hold out a validation set.
- **Bad data in, bad behaviour out** — a few hundred high-quality examples often beat thousands of noisy ones.
- **Safety regression** — fine-tuning can weaken a model's safety training; re-test after tuning.

Part 10 — Pretraining vs fine-tuning

Pretraining is a one-time, massive investment that gives a model general knowledge; fine-tuning is a cheap, repeatable step that adapts that model to a specific job. Very few organisations pretrain; many fine-tune.

Aspect	Pretraining	Fine-tuning
Goal	General language understanding and world knowledge	Specific behaviour, task, style or domain
Starting point	Random weights	A pretrained model
Data	Trillions of unlabelled tokens	Thousands to millions of curated, often labelled examples
Supervision	Self-supervised (next or masked token)	Supervised pairs, human preferences, or verifiable rewards
Compute	Thousands of GPUs for weeks to months	One to a few GPUs for hours to days (with PEFT)
Cost	Millions to hundreds of millions of dollars	Tens to thousands of dollars
Who does it	Frontier labs and large tech companies	Startups, enterprises, researchers, individuals
Output	Base model	Instruction, chat, domain or task model
Analogy	A broad school and university education	On-the-job training for a specific role

Prompting vs RAG vs fine-tuning

Fine-tuning is not the only way to customise a model, and often not the first choice.

Need	Best approach	Why
Better format, tone or step-by-step answers	Prompt engineering	Instant, free, no training
Answers from private or frequently changing documents	RAG	Knowledge updates without retraining; answers can cite sources
A consistent style, structured output or narrow task at high volume	Fine-tuning (LoRA)	Bakes behaviour in; shorter prompts, lower cost per call
A smaller, cheaper model that matches a big one on one task	Fine-tuning via distillation	Train the small model on the big model's outputs
Deep domain language (medicine, law) unseen in training	Domain-adaptive pretraining + fine-tuning	Teaches new vocabulary and concepts

A good order to try: **prompt** → **RAG** → **fine-tune**, and combine them — many production systems use a fine-tuned model *with* RAG.

Part 11 — Modern refinements

The core 2017 design is still recognisable in today's models, but a set of engineering refinements makes them faster, cheaper and able to read far longer inputs.

Technique	Problem it solves	How it works	Used in
KV cache	Re-computing attention for all earlier tokens at every generation step	Store each token's keys and values once; new tokens reuse them	Every LLM at inference
Grouped-query attention (GQA)	The KV cache uses huge memory at long context	Many query heads share a few key/value heads (Part 5)	Llama 2 70B+, Llama 3, Mistral
FlashAttention	Attention is limited by slow GPU memory reads	Computes attention in tiles inside fast on-chip memory, never storing the full n × n matrix	Most training and serving stacks
RoPE + context scaling	Models fail beyond their trained length	Rotary positions, stretched or interpolated to extend context	Llama 3.1 (128k), Qwen, many others

Technique	Problem it solves	How it works	Used in
Sparse / sliding-window attention	Quadratic cost of full attention	Each token attends only to nearby tokens or a chosen subset	Mistral 7B, Longformer
Mixture of Experts (MoE)	Bigger models cost more per token	Replace one FFN with many "experts"; a router sends each token to only 1–2 of them	Mixtral, DeepSeek-V3, reportedly several frontier models
RMSNorm, SwiGLU	Small efficiency and quality gains	Simpler normalisation; gated FFN activation	Llama, PaLM, Mistral
Quantisation	Memory and cost at inference	Store weights in 8-, 4- or fewer bits instead of 16	Local and edge deployment (GGUF, GPTQ, AWQ)
Speculative decoding	Token-by-token generation is slow	A small draft model proposes several tokens; the big model checks them in one pass	Many production serving systems

Beyond text

- **Vision Transformers (ViT)** cut an image into patches (e.g. 16 × 16 pixels) and treat each patch as a token.
- **Multimodal LLMs** project image, audio and text tokens into one shared embedding space, so a single Transformer can read all of them.
- **Diffusion Transformers (DiT)** replace the U-Net in image and video diffusion models with a Transformer.

Glossary and self-check

Glossary

Term	Meaning
Attention head	One independent attention calculation with its own Q, K, V weights
Base model	A pretrained model before instruction tuning
BPE	Byte-Pair Encoding — builds a subword vocabulary by merging frequent symbol pairs
Causal mask	Blocks attention to future tokens in decoders
Context window	Maximum number of tokens the model can process at once

Term	Meaning
Cross-entropy loss	Measures how much probability the model gave the correct next token
d_model	Size of each token's vector inside the model
Embedding	Learned vector representing a token's meaning
FFN	Feed-forward network applied to each token inside every block
KV cache	Stored keys and values reused during generation
Layer norm	Rescales vectors to keep training stable
Logits	Raw scores for each vocabulary token before softmax
LoRA	Fine-tuning method that trains small low-rank matrices beside frozen weights
MLM	Masked language modelling — BERT's pretraining task
Perplexity	e^loss; lower means the model predicts text better
Positional encoding	Adds word-order information to embeddings
Q, K, V	Query, Key, Value projections used in attention
Residual connection	Adds a layer's input to its output (x + f(x))
RLHF / DPO	Methods that tune models using human preference data
RoPE	Rotary positional embedding, encoding relative position by rotating Q and K
SFT	Supervised fine-tuning on prompt–response examples
Softmax	Converts scores into probabilities that sum to 1
Token	A unit of text (word or word piece) mapped to an integer ID
Vocabulary (V)	The full set of tokens a tokenizer knows

Self-check quiz

1. Why do LLMs use subword tokenization rather than whole words or characters? *(Part 2)*
2. Using the BPE table, how would *lowest* be tokenized, and why? *(Part 2)*
3. What is the shape of the embedding table, and how is an embedding looked up? *(Part 3)*
4. Why does a Transformer need positional encoding at all? *(Part 3)*
5. Explain the role of Query, Key and Value in one sentence each. *(Part 4)*
6. Why are scores divided by $\sqrt{d_k}$ before softmax? *(Part 4)*
7. What does the causal mask prevent, and which models don't use one? *(Part 4)*

8. In GPT-2 small, how big is each attention head and how many are there? *(Part 5)*
9. What are the two sub-layers of a Transformer block, and what do residual connections do? *(Part 6)*
10. Estimate the parameters of a model with L = 32 and d = 4,096. *(Part 7 — about 6.4 billion plus embeddings)*
11. Compare the pretraining objectives of BERT, GPT and T5. *(Part 8)*
12. What did the Chinchilla study conclude? *(Part 8)*
13. List the three steps of RLHF. *(Part 9)*
14. Why does LoRA need so much less memory than full fine-tuning? *(Part 9)*
15. You need a chatbot that answers from a policy manual updated monthly. Fine-tune or RAG? *(Part 10)*
16. What does a KV cache store, and why does GQA shrink it? *(Part 11)*